

Enterprise Kubernetes Observability, Monitoring, Alerting, Backup & Recovery Guide

Designed for the Church Management Microservices Platform built with React, Fastify, Kubernetes, Amazon RDS PostgreSQL, Prometheus, Grafana, Elasticsearch, Kibana, Filebeat and Alertmanager.

Executive Summary

This document explains not only what technologies are used but why they are used, what problem they solve, where they fit into the architecture, and how to implement them step-by-step. The goal is production-grade observability and operational excellence.

Why Observability Matters

Monitoring tells you something is wrong. Observability helps you understand why it is wrong. For a church platform managing members, events, finance records, sacraments, notifications and administration, downtime or data loss can directly affect operations. The observability stack provides visibility into infrastructure, containers, applications, databases, logs and user-facing services.

Architecture Overview

Users access a React frontend. Requests pass through ingress to Fastify APIs and business microservices. Data is stored in Amazon RDS PostgreSQL. Metrics are collected by Prometheus. Dashboards are presented in Grafana. Logs are shipped through Filebeat to Elasticsearch and visualized in Kibana. Alerts are routed through Alertmanager to Slack.

Why Kubernetes

Kubernetes provides deployment automation, self-healing, rolling updates, service discovery, scaling, resource management and workload isolation. Instead of manually managing servers, the platform manages application lifecycles automatically.

Why Prometheus

Prometheus is the industry standard for Kubernetes metrics. It scrapes metrics from nodes, containers and applications. It stores time series data and evaluates alert rules. Prometheus answers questions such as: Is CPU increasing? Which service is slow? How many requests occurred during worship registration?

Why Grafana

Grafana transforms raw metrics into operational dashboards. Leadership can see service health while administrators can analyze trends, latency, errors and capacity utilization.

Why Filebeat

Containers are temporary. Logs disappear when containers terminate. Filebeat runs as a DaemonSet so every node automatically forwards logs to Elasticsearch, ensuring centralized log retention.

Why Elasticsearch and Kibana

Logs are useful for root-cause analysis. Elasticsearch indexes logs for fast searching while Kibana provides visualization and querying. Engineers can quickly answer: Which user action produced the error? Which API failed?

Why Node Exporter

Node Exporter exposes CPU, memory, filesystem, load and network metrics. These metrics reveal infrastructure health regardless of application behavior.

Why cAdvisor

cAdvisor provides container-level visibility such as container CPU usage, memory consumption, filesystem usage and restart behavior.

Why kube-state-metrics

Node metrics are not enough. kube-state-metrics exposes Kubernetes object information such as deployment status, pod health, replica availability and failed jobs.

Why Alertmanager

Dashboards are useful only when someone is watching them. Alertmanager proactively sends notifications when predefined conditions occur.

Step 1 - Deploy Cluster Monitoring

Install kube-prometheus-stack using Helm. This deploys Prometheus, Grafana, Alertmanager and kube-state-metrics. Verify scraper targets are healthy from the Prometheus UI.

Step 2 - Instrument Fastify

Integrate prom-client. Expose a /metrics endpoint. Collect request counts, duration, status codes, error totals and business metrics. Business metrics can include new member registrations, attendance submissions or donations processed.

Step 3 - Logging Implementation

Use Pino structured logging. Write logs to stdout. Deploy Filebeat as a DaemonSet. Configure Elasticsearch output. Build Kibana dashboards and saved searches for critical services.

Step 4 - Dashboard Design

Create Infrastructure, Kubernetes, Application and Database dashboards. Infrastructure shows CPU, disk and memory. Kubernetes shows pods, deployments and restarts. Application shows request rate, latency and errors. Database shows connections, throughput and storage growth.

Step 5 - Alert Design

Create alerts for high CPU, memory exhaustion, pod restart storms, database connectivity failures, high latency, elevated 5xx error rates, disk saturation and unavailable replicas.

Amazon RDS PostgreSQL Strategy

Use RDS instead of self-managed PostgreSQL. AWS manages backups, patching, monitoring and recovery. This reduces operational complexity and increases reliability.

Backup Strategy Layer 1 – Automated Backups

Enable automated backups and point-in-time recovery. This protects against accidental record deletion or application defects. Recommended retention is 7 to 30 days depending on business requirements.

Backup Strategy Layer 2 – Manual Snapshots

Create snapshots before schema migrations, major releases, database upgrades or bulk imports. Name snapshots consistently and document restoration procedures.

Backup Strategy Layer 3 – Logical Backups

Schedule pg_dump exports and store them in Amazon S3. Logical backups provide portability and can be restored into different environments such as development, testing or disaster recovery.

Recovery Planning

Document RPO and RTO. Define who performs recovery, how recovery is verified, communication procedures, escalation paths and post-incident review requirements.

Restore Scenario A

Accidental deletion: perform point-in-time restore to a temporary instance, validate recovery and restore required records.

Restore Scenario B

Corruption: restore latest snapshot, validate application functionality and redirect traffic.

Restore Scenario C

Regional migration or environment rebuild: restore pg_dump backups into a replacement RDS instance.

Security Controls

Implement RBAC, TLS, Secrets Manager, encrypted storage, network policies, IAM least privilege and audit logging. Monitoring data should be protected because it may reveal system architecture and operational information.

Production Readiness Checklist

Monitoring enabled, logging enabled, alerting configured, backup validation completed, restore testing completed, dashboards documented, runbooks created, resource limits configured, secrets secured and disaster recovery procedures documented.

Portfolio Narrative

This project demonstrates cloud architecture, Linux administration, Kubernetes operations, observability engineering, DevOps practices, AWS infrastructure, database reliability engineering, backup strategy design, logging pipelines, metrics engineering and operational readiness.

Reference Architecture

```
Users
|
React Frontend
|
Ingress
|
Fastify API Gateway
|
Microservices
|
Amazon RDS PostgreSQL

Logging
-----
Pino -> stdout -> Filebeat(DaemonSet) -> Elasticsearch -> Kibana

Metrics
-----
Node Exporter(DaemonSet)
cAdvisor
kube-state-metrics
Fastify /metrics
    |
    Prometheus
    |
    Grafana

Alerting
-----
Prometheus -> Alertmanager -> Slack
```